

Projet de NFE204 Bases de données avancées (1)
Transformation d'une base de données relationnelle
Vers
Une base de données NoSQL

Contenu

Introduction	3
L'application actuelle	3
Présentation.....	3
Fonctionnement.....	3
Collecte et importation des données.....	5
La base de données relationnelle	7
Présentation.....	7
Les requêtes	8
Requête 1	8
Requête 2	8
Requête 3	9
Requête 4	10
Le NoSQL	11
Présentation.....	11
Solution retenue	11
Installation de MongoDB	14
La base de données NoSQL.....	17
L'architecture	17
Le langage de manipulation de données dans MongoDB	18
La fonction Aggregate	19
La fonction MapReduce	20
Les requêtes	22
Requête 1	24
Requête 2	25
Requête 3	25
Requête 4	26
Les tests de performance.....	27
Importation des données.....	27
Recherche d'un logiciel	27
Modification de l'affichage	27
Tri sur le nom du logiciel.....	27
Navigation dans les pages.....	27
Résultats.....	27
Conclusion.....	28

Introduction

Depuis plusieurs années, les bases de données NoSQL font de plus en plus parler d'elles. Ce type de base est destiné à être utilisé pour traiter de grand volume de données dans des environnements distribués.

Les bases NoSQL sont-elles capable de travailler sur des volumes de données beaucoup moins important tout en conservant un niveau de performance acceptable. Sont-elles facile à intégrer dans une architecture existante.

Le but de mon projet est de transformer une application web reposant sur une base de données relationnelle vers une base de données NoSQL et de voir si les performances sont au rendez-vous.

L'application actuelle

Présentation

Dans l'administration où je travaille, nous avons 2 principaux réseaux qui sont physiquement séparés.

Un premier réseau appelé "INTRANET" qui est un réseau à vocation bureautique avec un niveau de confidentialité qui lui impose d'être isolé du monde extérieur, composé d'environ 500 postes clients. Un deuxième réseau, appelé "INTERNET", composé d'environ 300 postes clients, qui permet d'accéder à l'INTERNET.

Ces 2 réseaux sont essentiellement composés de serveurs Microsoft Windows et de postes clients fonctionnant sous Windows XP ou Windows 7. Ces 2 réseaux sont configurés en domaine active directory.

L'officier de sécurité des systèmes d'information qui est en charge d'assurer la sécurité de l'ensemble de nos systèmes d'information, m'a demandé d'établir un inventaire logiciel des réseaux INTRANET et INTERNET.

Pour présenter cet inventaire de façon assez conviviale, facile à lire, et simple d'accès, je choisis de réaliser une interface web ([figure 1](#) et [figure 2](#)).

Pour réaliser cette interface web, j'utilise une architecture 3 tiers composée d'un serveur web Apache, d'une base de données relationnelle SQL Serveur 2008 R2, du langage serveur PHP, de la bibliothèque JavaScript JQuery, et du plugin JQuery Datatable

Fonctionnement

A partir de la page d'accueil l'utilisateur peut sélectionner le réseau qu'il souhaite visualiser **(1)**.

Ensuite il a à sa disposition 2 onglets **(2)**

- Un premier onglet qui permet d'afficher un tableau contenant la liste des logiciels avec le nom, la version et le nombre d'installation **(3)**. Lorsque l'utilisateur clique sur une ligne de ce tableau, il apparait sur la droite un tableau contenant la liste des machines où est installé le logiciel sélectionné **(4)**.
- Un deuxième onglet qui permet d'afficher un tableau contenant la liste des machines avec le nom, la date d'inventaire et le nombre de logiciels installés sur la machine **(5)**. Lorsque l'utilisateur clique sur une ligne de ce tableau, il apparait sur la droite un tableau contenant la liste des logiciels installés sur la machine sélectionnée **(6)**.

L'utilisateur peut jouer sur l'affichage dans chaque tableau. Il peut sélectionner le nombre d'enregistrements affichés par page **(7)**, il peut changer de page par l'intermédiaire de la barre de défilement **(8)**, et il peut rechercher un logiciel ou une machine **(9)**.

INTRANET INTERNET (1)

INTRANET - PAR LOGICIELS INTRANET - PAR MACHINES (2)

Rapport de synthèse du réseau INTRANET PAR LOGICIELS Export CSV
717 machines analysées, 731 logiciels détectés

Liste des machines possédant 32 Bit HP CIO Components Installer

(7) Afficher 25 logiciels (3) Rechercher un logiciel : (9)

Logiciel	Version	Nombre
"Minimal SYstem 1.0.11"	1.0.11	44
32 Bit HP CIO Components Installer	8.1.4	5
32 Bit HP CIO Components Installer	3.1.1	71
3D for GeoConcept (2.9.11)	2.9.11	1
64 Bit HP CIO Components Installer	15.2.1	3
64 Bit HP CIO Components Installer	13.2.1	1
64 Bit HP CIO Components Installer	8.2.4	26
64 Bit HP CIO Components Installer	8.2.2	39
64 Bit HP CIO Components Installer	3.2.1	194
64 Bit HP CIO Components Installer	4.2.1	244
7-Zip 4.57	inconnue	24
7-Zip 4.65	inconnue	10
7-Zip 9.20	9.20.00.0	152
7-Zip 9.20 (x64 edition)	9.20.00.0	563
7-Zip 9.22beta	inconnue	54

1 à 25 sur 731 (8) Début Précédent 1 2 3 4 5 Suivant Fin

Afficher 10 machines (4) Rechercher une machine :

Nom	Date installation
PC-ALLAIRE	19/09/2011
PC-AUDY	14/06/2011
PC-AUGER	31/05/2010
PC-BANVILLE	02/11/2010
PC-BEAUMONT	10/12/2009
PC-BEDARD	05/09/2011
PC-BELZILE	18/11/2009
PC-BERTHIAUME	15/07/2011
PC-BLOUIN	02/09/2010
PC-BOILEAU	09/09/2009

1 à 10 sur 71 Début Précédent 1 2 3 4 5 Suivant Fin

Figure 1

INTRANET INTERNET

INTRANET - PAR LOGICIELS INTRANET - PAR MACHINES

Rapport de synthèse du réseau INTRANET PAR MACHINES
717 machines analysées, 731 logiciels détectés

Logiciels installés sur MVL-AUCLAIR

Afficher 10 machines (5) Rechercher une machine :

Machine	Date inventaire	Logiciels
MVL-AUCLAIR	17/12/2013	59
MVL-BEAUCHEMIN	17/12/2013	59
MVL-BELLEMARE	17/12/2013	60
MVL-BRIERE	17/12/2013	59
MVL-BRISEBOIS	17/12/2013	49
MVL-BROWN	17/12/2013	59
MVL-CADIEUX	17/12/2013	59
MVL-DUBUC	17/12/2013	59
MVL-DUROCHER	2/12/2013	49
MVL-DUSSAULT	17/12/2013	59

1 à 10 sur 717 Début Précédent 1 2 3 4 5 Suivant Fin

Afficher 10 logiciels (6) Rechercher un logiciel :

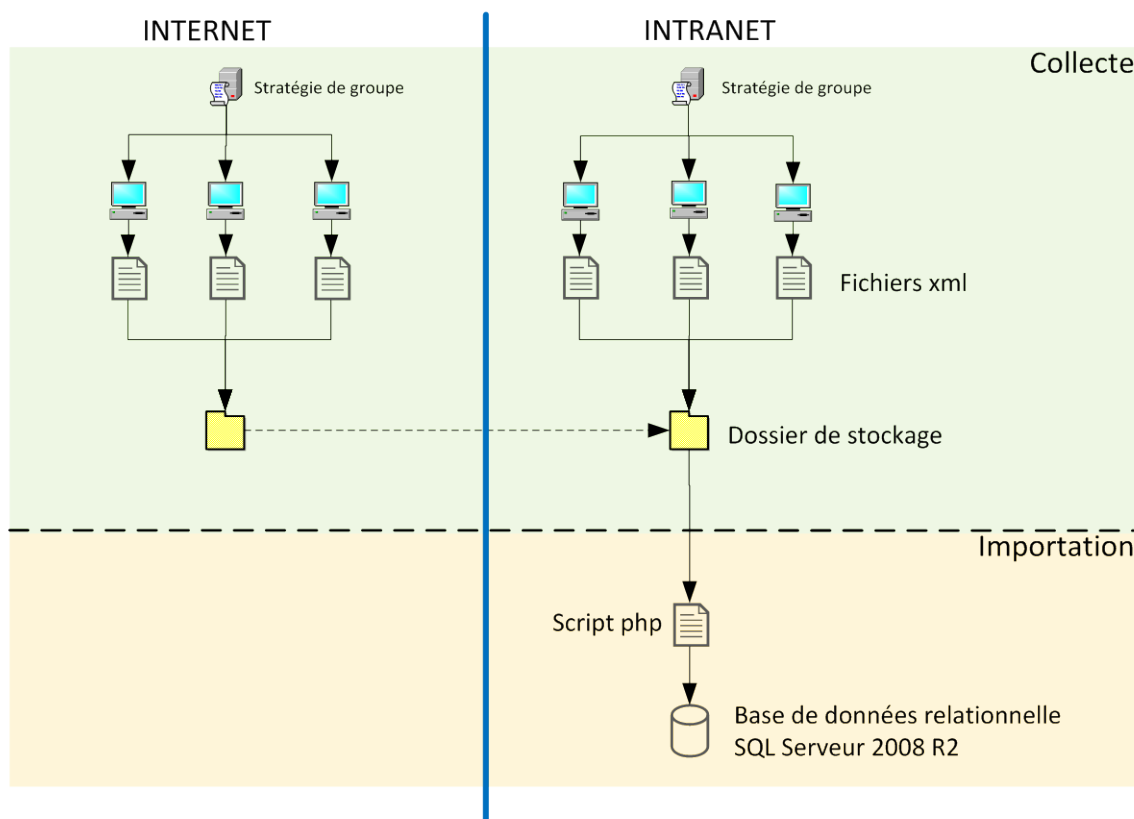
Logiciel	Version	Date installation
7-Zip 9.20 (x64 edition)	9.20.00.0	13/09/2012
Acid_7	1.0	12/06/2013
Adobe Reader X (10.1.4) - Français	10.1.4	13/09/2012
AuthentiC Webpack v4.4.2 64-bit	4.4.2	13/09/2012
Certificat_AC_Interne	1.0	08/11/2013
Configuration Manager Client	5.00.7711.0000	12/06/2013
Export ACID	1.0.2	12/06/2013
F-Secure Anti-Virus for Workstations - Protection virus et logiciels espions	9.50.19220	inconnue
Intel(R) Network Connections Drivers	16.8	inconnue
Intel(R) Processor Graphics	9.17.10.2867	inconnue

1 à 10 sur 59 Début Précédent 1 2 3 4 5 Suivant Fin

Figure 2

Collecte et importation des données

Le schéma ci-dessous montre le mécanisme de collecte et d'importation des données stockées dans la base de données SQL Serveur 2008 R2 et affichées dans l'interface web.



Les données sont collectées via une stratégie de groupe qui exécute sur chaque machine un script permettant d'obtenir l'ensemble des logiciels installés sur celle-ci. Un fichier xml est généré par machine.

Ce fichier xml est composé de 2 blocs, un bloc `<ordinateur></ordinateur>` contenant toutes les informations de la machine et un bloc `<logiciels></logiciels>` contenant la liste des logiciels, comme ci-dessous.

```
<inventaire date="20/12/2013">
```

```

<ordinateur>
  <nom>INET-ASSELIN</nom>
  <marque>Dell Inc.</marque>
  <modele>OptiPlex 790</modele>
  <ram>3977</ram>
  <os nom="Microsoft Windows 7 Professionnel" version="6.1.7601" sp="Service Pack 1"/>
  <reseau>inter</reseau>
  <adresse_mac>18:03:73:48:57:C9</adresse_mac>
</ordinateur>

<logiciels>
  <logiciel nom="Package de pilotes Windows - Dell Inc. PBADRV System (09/11/2009 1.0.1.6)" date_installation=""
editeur="Dell Inc." version="09/11/2009 1.0.1.6"/>
  <logiciel nom="Intel(R) Graphics Media Accelerator Driver" date_installation="" editeur="Intel Corporation"
version="8.15.10.1930"/>
  ...
  <logiciel nom="Update for Microsoft Filter Pack 2.0 (KB2810071) 32-Bit Edition" date_installation=""
editeur="Microsoft" version=""/>
  <logiciel nom="Security Update for Microsoft Office 2010 (KB2826035) 32-Bit Edition" date_installation=""
editeur="Microsoft" version=""/>
  ...
</logiciels>
</inventaire>

```

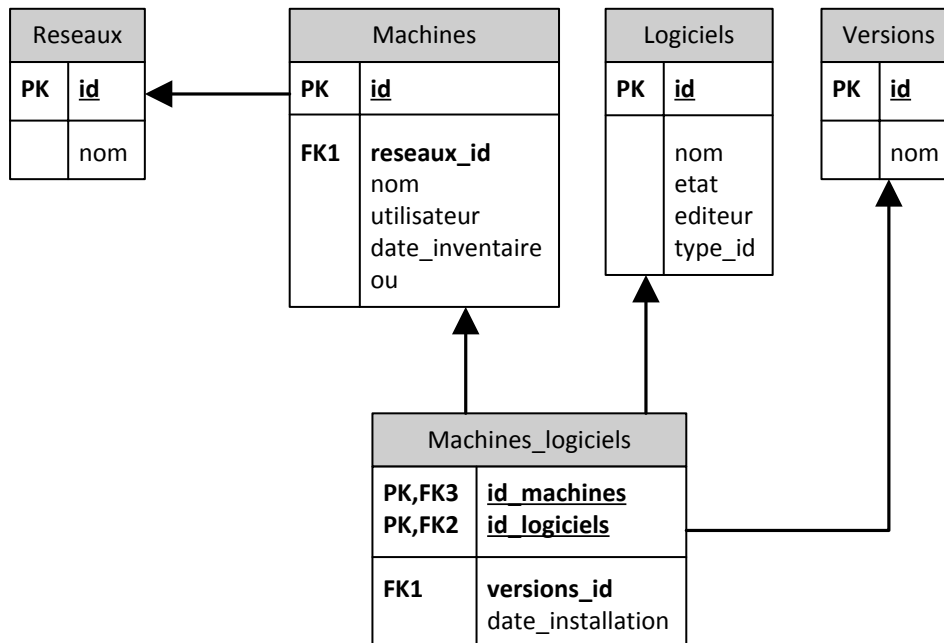
Ces fichiers sont stockés dans un dossier sur le serveur hébergeant l'interface web. Les 2 réseaux étant physiquement isolés, je dois copier manuellement les fichiers xml obtenus sur le réseau INTERNET dans le dossier de stockage sur le réseau INTRANET.

Pour terminer, je lance à partir de l'interface web un script PHP permettant d'importer dans la base de données le contenu des fichiers xml. Le script lit les fichiers xml et retravaille les données pour qu'elles correspondent au schéma relationnel

La base de données relationnelle

Présentation

La structure de la base de données est assez simple, elle est composée de 5 tables, dont le schéma relationnel est le suivant.



- **Machines.** Cette table contient la liste de toutes les machines présentes sur les réseaux INTRANET et INTERNET. Chaque enregistrement spécifie notamment l'identifiant unique de la machine (id), son nom (nom), son réseau d'appartenance (reseaux_id) et sa dernière date d'inventaire (date_inventaire).
- **Reseaux.** Cette table contient la liste des réseaux. Chaque enregistrement spécifie l'identifiant unique du réseau (id) et son nom (nom).
- **Logiciels.** Cette table contient la liste des logiciels présents sur les 2 réseaux. Chaque enregistrement spécifie notamment l'identifiant unique du logiciel (id), son nom (nom), son éditeur (editeur) et son type (type_id). Le champ type_id sert à différencier un patch de sécurité Microsoft (0) et un logiciel classique (1).
- **Versions.** Cette table contient la liste des diverses versions des logiciels. Chaque enregistrement spécifie l'identifiant unique de la version (id) et son nom (nom).
- **Machines_logiciels.** Cette table contient la liste de tous les logiciels installés sur une machine avec sa version ainsi que sa date d'installation. Chaque enregistrement spécifie l'identifiant de la machine (id_machines), l'identifiant du logiciel (id_logiciels), l'identifiant de la version (versions_id) et la date d'installation du logiciel (date_installation).

Les requêtes

A partir de cette base de données j'utilise 4 requêtes pour générer le contenu des 4 tableaux composant l'interface web.

Requête 1

```
select l.id as logiciel_id , v.id as version_id , l.nom as logiciel_nom, v.nom as
logiciel_version, COUNT(ml.id_logiciels) as nb_logiciels
from logiciels l
left join machines_logiciels ml on ml.id_logiciels=l.id
left join versions v on v.id=ml.versions_id
left join machines m on m.id=ml.id_machines
where l.etat=1 and l.type_id=0 and m.reseaux_id=0
group by l.id,v.id,l.nom,v.nom
order by logiciel_nom
```

Elle permet d'obtenir la liste des logiciels avec le nom, la version et le nombre d'installation du logiciel. Par défaut, le résultat est trié sur le nom du logiciel. Ci-dessous le tableau généré grâce à cette requête.

Afficher 25	logiciels	Rechercher un logiciel :	
Logiciel	Version	Nombre	
"Minimal SYStem 1.0.11"	1.0.11	44	
32 Bit HP CIO Components Installer	8.1.4	5	
32 Bit HP CIO Components Installer	3.1.1	71	
3D for GeoConcept (2.9.11)	2.9.11	1	
64 Bit HP CIO Components Installer	15.2.1	3	
64 Bit HP CIO Components Installer	13.2.1	1	
64 Bit HP CIO Components Installer	8.2.4	26	
64 Bit HP CIO Components Installer	8.2.2	39	
64 Bit HP CIO Components Installer	3.2.1	194	
64 Bit HP CIO Components Installer	4.2.1	244	
7-Zip 4.57	inconnue	24	
7-Zip 4.65	inconnue	10	
7-Zip 9.20	9.20.00.0	152	
7-Zip 9.20 (x64 edition)	9.20.00.0	563	
7-Zip 9.22beta	inconnue	54	
1 à 25 sur 731			Début Précédent 1 2 3 4 5 Suivant Fin

Requête 2

```
select m.id as machine_id, m.nom as machine_nom, m.date_inventaire as date_inventaire,
COUNT(ml.id_logiciels) as nb_logiciels
from Machines m
left join Machines_Logiciels ml on ml.id_machines=m.id
left join Logiciels l on l.id=ml.id_logiciels
where m.reseaux_id=0 and l.type_id=0
group by m.id,m.nom,m.date_inventaire
```

Elle permet d'obtenir la liste des machines avec le nom, la date d'inventaire et le nombre de logiciels installés sur la machine. Par défaut, le résultat est trié sur le nom de la machine par ordre alphabétique. Ci-dessous le tableau généré grâce à cette requête.

Afficher 25 machines		Rechercher une machine :	
Machine	Date inventaire	Logiciels	
MVL-AUCLAIR	17/12/2013	59	
MVL-BEAUCHEMIN	17/12/2013	59	
MVL-BELLEMARE	17/12/2013	60	
MVL-BRIERE	17/12/2013	59	
MVL-BRISEBOIS	17/12/2013	49	
MVL-BROWN	17/12/2013	59	
MVL-CADIEUX	17/12/2013	59	
MVL-DUBUC	17/12/2013	59	
MVL-DUROCHER	2/12/2013	49	
MVL-DUSSAULT	17/12/2013	59	
MVL-ETHIER	19/12/2013	49	
MVL-GARNEAU	17/12/2013	59	
MVL-GAUVIN	17/12/2013	59	
MVL-GOUBOUT	17/12/2013	59	
MVL-GUILBAULT	17/12/2013	59	
1 à 25 sur 717		Début	Précédent 1 2 3 4 5 Suivant Fin

Requête 3

```

select l.nom as logiciel_nom,ml.date_installation as date_installation,v.nom as
logiciel_version
from Logiciels l
left join Machines_Logiciels ml on ml.id_logiciels=l.id
left join Machines m on m.id=ml.id_machines
left join versions v on v.id=ml.versions_id
where m.id=330 and l.type_id=0
order by logiciel_nom

```

Elle permet d'obtenir, pour une machine sélectionnée, la liste des logiciels installés sur celle-ci avec le nom, la version et la date d'installation. Par défaut, le résultat est trié sur le nom du logiciel par ordre alphabétique. Ci-dessous le tableau généré grâce à cette requête.

Afficher 25 logiciels		Rechercher un logiciel :	
Logiciel	Version	Date installation	
7-Zip 9.20 (x64 edition)	9.20.00.0	13/09/2012	
Acid_7	1.0	21/06/2013	
Adobe Reader X (10.1.4) - Français	10.1.4	13/09/2012	
Authentic Webpack v4.4.2 64-bit	4.4.2	13/09/2012	
Certificat_AC_Interne	1.0	21/06/2013	
Configuration Manager Client	5.00.7711.0000	21/06/2013	
Export ACID	1.0.2	21/06/2013	
Intel(R) Network Connections Drivers	16.8	inconnue	
Intel(R) Processor Graphics	9.17.10.2867	inconnue	
ISIS Wrapper PKCS11	1.0.2	21/06/2013	
Lumension Endpoint Security Client	4.4.1447	21/06/2013	
Microsoft .NET Framework 4 Client Profile	4.0.30319	21/06/2013	
Microsoft .NET Framework 4 Client Profile FRA Language Pack	4.0.30319	20/09/2012	
Microsoft Office 2010 Service Pack 1 (SP1)	inconnue	inconnue	
Microsoft Office Access MUI (French) 2010	14.0.6029.1000	21/02/2013	
1 à 25 sur 49		Début	Précédent 1 2 Suivant Fin

Requête 4

```
select m.nom,ml.date_installation from Machines m
left join Machines_Logiciels ml on ml.id_machines=m.id
where ml.id_logiciels=264 and ml.versions_id=78 and m.reseaux_id=0
order by m.nom
```

Elle permet d'obtenir, pour un logiciel sélectionné, la liste des machines où est installé ce logiciel avec le nom, et la date d'installation. Par défaut, le résultat est trié sur le nom de la machine par ordre alphabétique. Ci-dessous le tableau généré grâce à cette requête.

Afficher 25 machines	Rechercher une machine :
Nom	Date installation
PC-ALAIN	02/04/2013
PC-BEAUDET	02/04/2013
PC-BEAUSOLEIL	27/11/2012
PC-BELIVEAU	03/04/2013
PC-BOURDAGES	21/08/2013
PC-BRAULT	09/10/2013
PC-BRUNELLE	27/11/2012
PC-BUJOLD	12/12/2012
PC-CHALIFOUX	30/05/2013
PC-CHARRON	08/10/2013
PC-CHIASSON	02/04/2013
PC-COUSINEAU	09/10/2013
PC-DASILVA	03/06/2013
PC-DESMEULES	02/04/2013
PC-DOUCET	28/11/2012
1 à 25 sur 39	Début Précédent 1 2 Suivant Fin

Le NoSQL

Présentation

Les SGBD NoSQL sont normalement adaptés pour travailler sur un grand volume de données dans des environnements distribués. Ils abandonnent la notion de schéma de données que l'on trouve dans les SGBD relationnel.

La mise à l'échelle horizontale, ajout de serveurs pour augmenter les puissances de calculs et/ou l'espace de stockage, est beaucoup plus simple qu'avec une base de données relationnelle. En contrepartie de cette facilité, les SGBD NoSQL n'appliquent pas les principes ACID des transactions qui permettent de garantir une cohérence très forte des données. Ils appliquent le théorème CAP (Consistency, Availability, Partition tolérance) des environnements distribués qui dit qu'il est impossible à un instant T d'avoir les 3 propriétés respectées, seulement 2 sont possibles.

Il existe beaucoup de SGBD NoSQL, le site <http://nosql-database.org/> en fournit une liste exhaustive.

Solution retenue

Pour réaliser mon projet de transformation de ma base de données relationnelle en base NoSQL, j'ai choisi le moteur de bases de données MongoDB.

MongoDB fonctionne sur un serveur Windows et s'intègre facilement dans mon architecture existante grâce au pilote PHP. A ma connaissance c'est le seul moteur NoSQL où un pilote PHP est fourni. Au final mon architecture Apache/PHP/Jquery/SQL Serveur 2008 R2 va devenir une architecture Apache/PHP/JQuery/MongoDB.

Pour commencer, un peu de vocabulaire. En relationnel, nous parlons de **table** et d'**enregistrements**. Dans MongoDB, les tables deviennent des **collections** et les enregistrements des **documents**. Une collection peut contenir un nombre quelconque de documents. La structure d'un document peut varier d'un document à l'autre.

MongoDB est une base de données NoSQL orientée documents. Comme toute base NoSQL, il n'y a pas de schéma prédéfini dès la création de la base de données comme en relationnel. Dans le tableau ci-dessous, nous pouvons voir qu'en relationnel chaque enregistrement a une structure bien établie, alors qu'en NoSQL, la structure peut varier d'un document à l'autre.

Relationnel			NoSQL												
<table><tr><th>Prénom</th><th>Nom</th><th>Ville</th></tr><tr><td>Gérard</td><td>Mentor</td><td>Paris</td></tr><tr><td>Alain</td><td>Terieur</td><td>Brest</td></tr><tr><td>Jean</td><td>Némard</td><td>Nice</td></tr></table>			Prénom	Nom	Ville	Gérard	Mentor	Paris	Alain	Terieur	Brest	Jean	Némard	Nice	<pre>{ "_id" : ObjectId("530658a6998b1944d042c129"), "Prénom" : "Gérard", "Nom" : "Mentor", "Ville" : "Paris" }, { "_id" : ObjectId("530658a6998b1944d042c12a"), "Prénom" : "Alain", "Nom" : "Terieur", "Rue" : "25, rue de l'abreuvoir", "Code postal" : "29200", "Ville" : "Brest" }, { "_id" : ObjectId("530658a6998b1944d042c12b"), "Prénom" : "Jean", "Nom" : "Némard", "Adresse" : { "Rue" : "14, allée des pissenlits", "Ville" : "Nice" } }</pre>
Prénom	Nom	Ville													
Gérard	Mentor	Paris													
Alain	Terieur	Brest													
Jean	Némard	Nice													

Orientée document signifie que la structure d'un document se compose de paires "clé":valeur. La clé est le nom du champ, la valeur son contenu. La valeur peut être du **texte**, un **nombre**, un **tableau**, un **document**, un **tableau de documents**, ou tout autre type reconnu par MongoDB. Chaque document est identifié par un champ obligatoire unique nommé **_id**. Ci-dessous, un exemple d'un document contenu dans une collection.

```
{
  "_id" : ObjectId("530655fc085ad81dd2f88fbf"),
  "prénom" : "Pedro",
  "age" : 27,
  "hobbies" : [
    "petanque",
    "curling"
  ],
  "telephones" : {
    "domicile" : "01.74.51.24.36",
    "professionnel" : "01.25.33.12.58",
    "fax" : "01.25.12.24.26"
  },
  "adresses" : [
    {
      "type" : "domicile",
      "numero" : 18,
      "rue" : "rue de Lappe",
      "code_postal" : "75011",
      "ville" : "Paris"
    },
    {
      "type" : "professionnelle",
      "rue" : "36 rue de Vaugirard",
      "code_postal" : "75006",
      "ville" : "Paris"
    }
  ]
}
```

Installation de MongoDB

L'installation de MongoDB sous Windows est facile à réaliser, il faut télécharger le fichier zip fournit sur le site <http://www.mongodb.org/downloads> dans sa version 64 bits. Il existe une version 32 bits, mais celle-ci ne peut pas gérer de base de données supérieure à 2 GB.

Je décompresse le fichier zip à l'emplacement où je souhaite installer MongoDB, dans mon cas ce sera dans e:\serveur_web, un dossier "mongodb-win32-x86_64-2008plus-2.4.8" est créé.

Pour plus de facilités pour la suite, je renomme ce dossier en mongodb. J'ajoute dans la variable Path de Windows le chemin e:\serveur_web\mongodb\bin.

Je crée les dossiers où seront stockés les bases de données et les logs. Le dossier pour les logs n'est pas indispensable mais il devient obligatoire pour la création d'un service Windows MongoDB.

Au final la structure des dossiers sera la suivante.

Dossier	Description
e:\serveur_web\mongodb	Dossier d'installation de MongoDB
e:\serveur_web\mongodb\databases	Dossier contenant les bases de données
e:\serveur_web\mongodb\logs	Dossier contenant les fichiers log de MongoDB

Tout est en place pour démarrer un serveur MongoDB.

Il suffit de lancer une fenêtre de commandes Windows puis d'exécuter la commande

```
mongod --dbpath E:\serveur_web\mongodb\databases
```

Le paramètre --dbpath n'est pas obligatoire, s'il n'est pas mis, MongoDB utilisera par défaut le dossier c:\data\db

Pour tester que le serveur fonctionne correctement, il faut lancer une deuxième fenêtre de commandes et saisir

```
mongo
```

Si le résultat de la commande affiche

```
C:\>mongo
MongoDB shell version: 2.4.9
connecting to: test
>
```

C'est que tout fonctionne correctement.

Pour éviter de faire cette manipulation à chaque fois que je souhaite lancer mon serveur MongoDB, je vais créer un service Windows.

Pour éviter de lancer la commande "mongod --dbpath E:\serveur_web\mongodb\databases" à chaque fois que je souhaite démarrer mon serveur MongoDB, je crée un service Windows.

Pour installer ce service, il faut créer un fichier de configuration qui contiendra au minimum l'option `logpath` qui indique l'emplacement du fichier de log. Il existe d'autres options qu'il est possible de mettre dans le fichier de configuration. La liste des options est disponible dans la documentation MongoDB <http://docs.mongodb.org/manual/reference/configuration-options/>.

Dans mon fichier de configuration "mongod.cfg", je rajoute également l'option `dbpath`. Au final le fichier contient

```
dbpath=E:\serveur_web\mongodb\databases
logpath=E:\serveur_web\mongodb\logs\mongod.log
```

Ensuite, je lance une fenêtre de commandes avec les privilèges administrateur, je me positionne dans le dossier `e:\serveur_web\mongodb\bin`, puis j'exécute la commande

```
E:\serveur_web\mongodb\bin>mongod --config e:\serveur_web\mongodb\mongod.cfg --install
```

Si le résultat de la commande affiche

```
Wed Nov 06 12:47:12.434 Trying to install Windows service 'MongoDB'
Wed Nov 06 12:47:12.641 Service 'MongoDB' (Mongo DB) installed with command line
'E:\serveur_web\mongodb\bin\mongod.exe --config mongod.cfg --service'
Wed Nov 06 12:47:12.642 Service can be started from the command line with 'net start MongoDB'
```

C'est que le service s'est correctement créé.

A partir de ce moment, le serveur MongoDB sera lancé automatiquement à chaque démarrage du serveur.

Pour finaliser l'intégration de MongoDB dans mon architecture, j'installe le pilote PHP qui me permettra d'interagir avec mes bases MongoDB depuis mes scripts PHP.

La page officielle où se procurer ces pilotes se trouve sur <https://github.com/mongodb/mongo-php-driver>. Sous Windows pour télécharger la version la plus récente se rendre sur <http://pecl.php.net/package/mongo>, pour les versions précédentes sur <https://s3.amazonaws.com/drivers.mongodb.org/php/index.html>.

Personnellement j'ai pris le fichier zip présent sur le site <https://s3.amazonaws.com/drivers.mongodb.org/php/index.html> contenant les fichiers dll pour toutes les versions de PHP décrits dans le tableau ci-dessous.

Fichier	Description
php_mongo-1.4.5-5.2-vc9.dll	Pilote pour PHP 5.2 version thread safe
php_mongo-1.4.5-5.2-vc9-nts.dll	Pilote pour PHP 5.2 version non thread safe
php_mongo-1.4.5-5.3-vc9.dll	Pilote pour PHP 5.3 version thread safe
php_mongo-1.4.5-5.3-vc9-nts.dll	Pilote pour PHP 5.3 version non thread safe
php_mongo-1.4.5-5.3-vc9-nts-x86_64.dll	Pilote pour PHP 5.3 version non thread safe 64 bits
php_mongo-1.4.5-5.3-vc9-x86_64.dll	Pilote pour PHP 5.3 version thread safe 64 bits
php_mongo-1.4.5-5.4-vc9.dll	Pilote pour PHP 5.4 version thread safe
php_mongo-1.4.5-5.4-vc9-nts.dll	Pilote pour PHP 5.4 version non thread safe
php_mongo-1.4.5-5.4-vc9-nts-x86_64.dll	Pilote pour PHP 5.4 version non thread safe 64 bits
php_mongo-1.4.5-5.4-vc9-x86_64.dll	Pilote pour PHP 5.4 version thread safe 64 bits
php_mongo-1.4.5-5.5-vc11.dll	Pilote pour PHP 5.5 version thread safe
php_mongo-1.4.5-5.5-vc11-nts.dll	Pilote pour PHP 5.5 version non thread safe
php_mongo-1.4.5-5.5-vc11-nts-x86_64.dll	Pilote pour PHP 5.5 version non thread safe 64 bits
php_mongo-1.4.5-5.5-vc11-x86_64.dll	Pilote pour PHP 5.5 version thread safe 64 bits

Pour installer le pilote, il suffit de mettre dans le dossier des extensions PHP le fichier dll correspondant à votre version de PHP puis de rajouter la ligne **extension=php_mongo-version.dll** dans le fichier php.ini. Redémarrer le serveur Apache pour que la configuration soit prise en compte et le tour est joué.

Maintenant que mon architecture Apache/PHP/MongoDB est en place et fonctionnelle, je passe au point le plus important de ce changement, la base de données NoSQL.

La base de données NoSQL

L'architecture

En NoSQL il n'y a pas de schéma de données prédéfini. Je choisis donc de créer une seule collection machines. Chaque document de cette collection correspond à un fichier xml d'inventaire d'une machine.

Fichier XML	Document de la collection
<pre> <inventaire date="20/12/2013"> <ordinateur> <nom>INET-ASSELIN</nom> <marque>Dell Inc.</marque> <modele>OptiPlex 790</modele> <ram>3977</ram> <os nom="Microsoft Windows 7 Professionnel" version="6.1.7601" sp="Service Pack 1"/> <reseau>inter</reseau> <adresse_mac>18:03:73:48:57:C9</adresse_mac> </ordinateur> <logiciels> <logiciel nom="Package de pilotes Windows - Dell Inc. PBADRV System (09/11/2009 1.0.1.6)" date_installation="" editeur="Dell Inc." version="09/11/2009 1.0.1.6"/> ... <logiciel nom="Update for Microsoft Filter Pack 2.0 (KB2810071) 32-Bit Edition" date_installation="" editeur="Microsoft" version=""/> ... </logiciels> </inventaire> </pre>	<pre> { "_id" : 15, "date_inventaire" : "20/12/2013", "ordinateur" : { "nom" : "INET-ASSELIN", "marque" : "Dell Inc.", "modele" : "OptiPlex 790", "os" : { "nom" : "Microsoft Windows 7 Professionnel", "sp" : "Service Pack 1", "version" : "6.1.7601" }, "reseau" : "inter", "adresse_mac" : "18:03:73:48:57:C9" }, "logiciels" : [{ "_id" : 191, "nom" : "Package de pilotes Windows - Dell Inc. PBADRV System (09/11/2009 1.0.1.6)", "date_installation" : "inconnue", "editeur" : "Dell Inc.", "version" : "09/11/2009 1.0.1.6", "type" : 0 }, ... { "_id" : 154, "nom" : "Update for Microsoft Filter Pack 2.0 (KB2810071) 32-Bit Edition", "date_installation" : "inconnue", "editeur" : "Microsoft", "version" : "inconnue", "type" : 1 }, ...] } </pre>

Si je compare ma base relationnelle à ma base NoSQL, je passe d'un schéma relationnel composé de 5 tables à une seule collection en NoSQL.

Relationnel	NoSQL
Machines(id, #reseaux_id, nom, utilisateur, date_inventaire) Reseaux(id, nom) Logiciels(id, nom, etat, editeur, type_id) Versions(id, nom) Machines_logiciels(#id_machines, #id_logiciels, #versions_id, date_installation)	Machines

Le langage de manipulation de données dans MongoDB

MongoDB propose les commandes find, insert, remove et update respectivement équivalente à select, insert, delete et update en SQL. Pour réaliser des requêtes d'interrogation du type SELECT ... FROM ... WHERE ... ORDER BY ..., la fonction find est en générale suffisante. Pour des requêtes SQL faisant appel à des fonctions d'agrégation telles que COUNT ou SUM avec un GROUP BY ou HAVING la fonction find ne suffit plus.

MongoDB propose 3 fonctions pour réaliser de telle requête, group, aggregate et mapReduce. Voici un résumé des caractéristiques de chaque fonction

group	aggregate	mapReduce
- Fonctions limitées - Très lent - Le résultat ne doit pas excéder la taille d'un document BSON qui est de 16 Mo - Ne fonctionne pas dans un environnement distribué	- Succession d'opérateurs appliqués sur une collection - Alternative à mapreduce s'il ne faut faire que des \$group, \$match, \$sort - Le résultat ne doit pas excéder la taille d'un document BSON qui est de 16 Mo - Fonctionne en environnement distribué	- Utilisé pour réaliser des traitements sur des grands volumes de données - Permet de créer de nouvelles collections - Fonctionne en environnement distribué

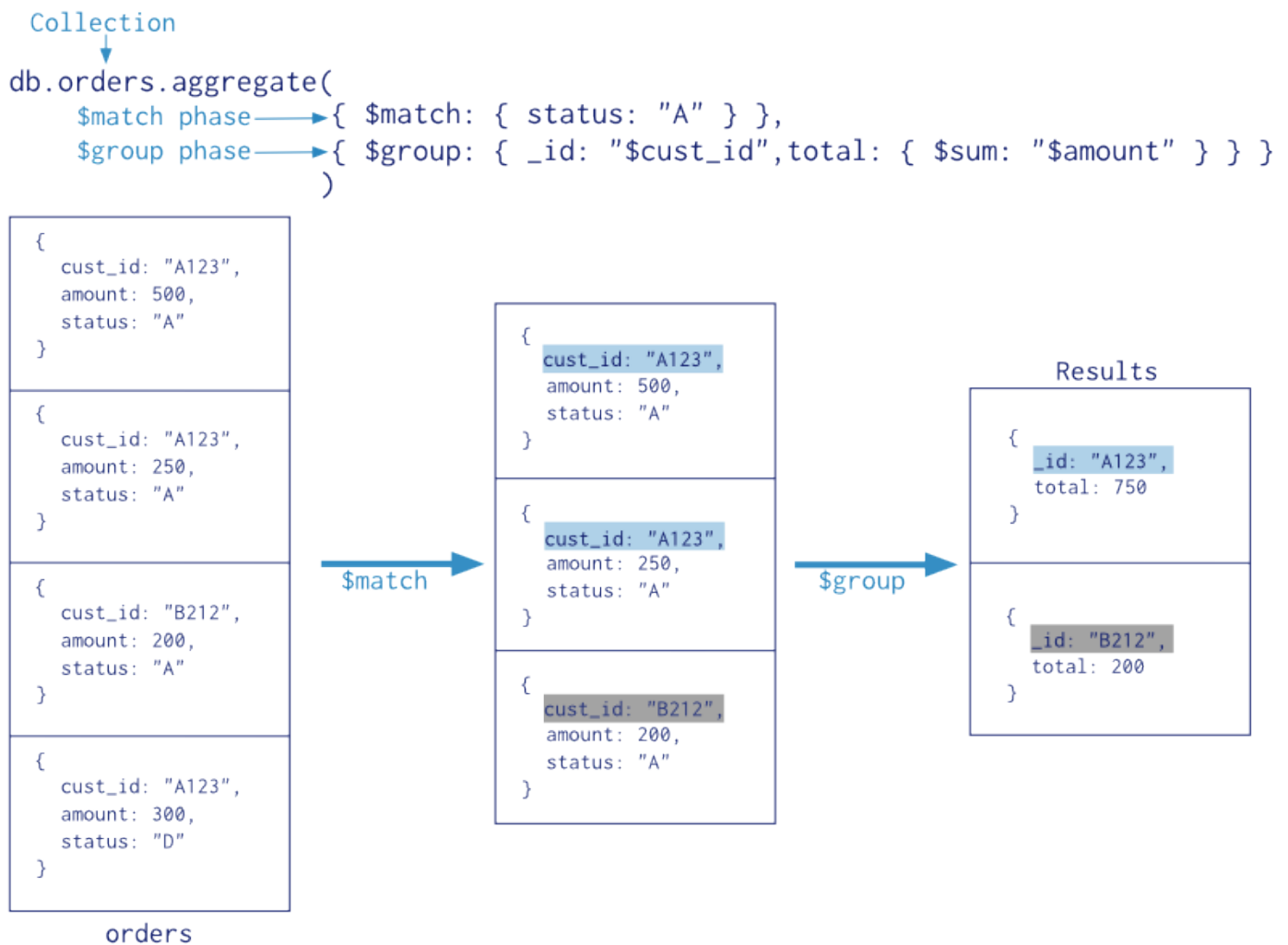
Dans mon projet j'ai uniquement utilisé les fonctions aggregate et mapReduce, group étant vraiment trop limité au niveau des fonctionnalités.

La fonction Aggregate

La syntaxe de la fonction aggregate est `db.collection.aggregate(pipeline)`. Le paramètre pipeline peut contenir les opérateurs suivant

- `$project` : équivalent à SELECT en SQL.
- `$match` : équivalent à WHERE en SQL.
- `$limit` : limiter le nombre de documents dans le pipeline.
- `$skip` : permet de faire un saut de x documents dans le pipeline et d'afficher le reste.
- `$unwind` : prend un tableau de documents et les retourne comme un flux de documents
- `$group` : équivalent à GROUP BY en SQL
- `$sort` : équivalent à ORDER BY en SQL
- `$geoNear` : Retourne un flux ordonné de documents basée sur la proximité à un point géospatiales.

Pour comprendre le comportement de cette fonction prenons l'exemple ci-dessous.



La première opération `{ $match: { status: "A" } }` est effectuée sur la collection, et son résultat est envoyé à la deuxième opération `{ $group: { _id: "$cust_id", total: { $sum: "$amount" } } }` et ainsi de suite tant qu'il y a des opérations dans le pipeline.

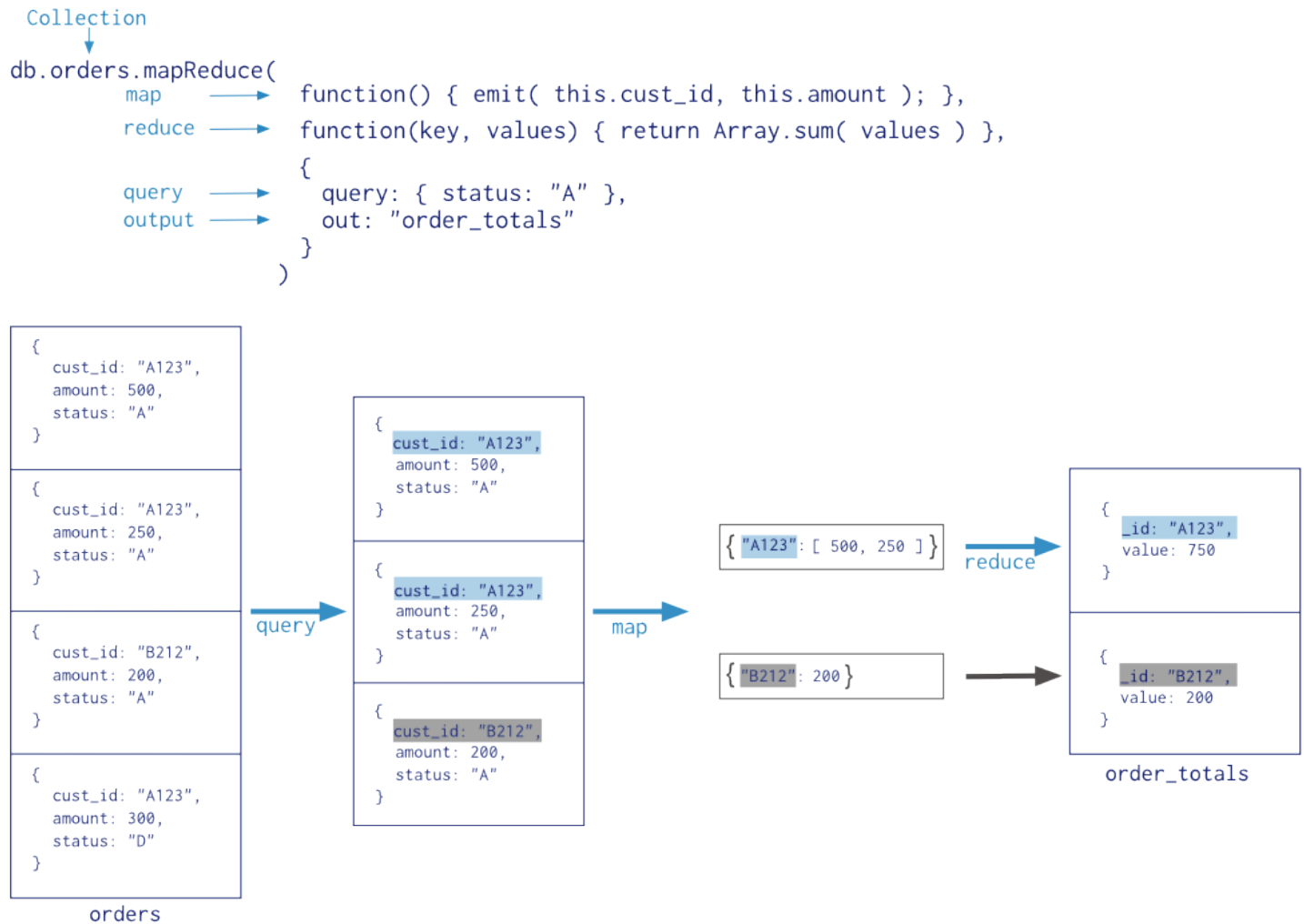
La fonction MapReduce

La syntaxe de la fonction mapReduce est la suivante

```
db.collection.mapReduce(  
  <map>,  
  <reduce>,  
  {  
    out: <collection>,  
    query: <document>,  
    sort: <document>,  
    limit: <number>,  
    finalize: <function>,  
    scope: <document>,  
    jsMode: <boolean>,  
    verbose: <boolean>  
  }  
)
```

Les explications détaillées sur toutes les options disponibles pour cette fonction sont disponible sur <http://docs.mongodb.org/manual/reference/method/db.collection.mapReduce/#db.collection.mapReduce>. Dans mon projet j'ai uniquement utilisé <map>, <reduce> et out. <map> et <reduce> sont les fonctions de mappage et de reduce, out permet de créer une nouvelle collection.

Pour comprendre le comportement de cette fonction, prenons l'exemple ci-dessous



Dans cet exemple est exécuté en premier sur la collection `query: {status:"A"}` qui permet de restreindre le nombre de documents sur lesquels la fonction map va travailler. Ensuite la fonction de mappage `function() {emit(this.cust_id, this.amount);}` entre en jeu. Elle va émettre des paires clés valeurs dont la clé sera le `cust_id` et la valeur sera soit un nombre si il y a un seul amount soit un tableau contenant les amount associés au `cust_id`. Pour terminer la fonction de reduce `function(key,values) {return Array.sum(values) }` va parcourir toutes les paires clés valeur générées par la fonction de mappage et faire la somme des amount. Le résultat final est retourné dans une nouvelle collection `order_totals`.

Les requêtes

Avec tous ces outils, je peux maintenant transformer mes 4 requêtes SQL en requête MongoDB. Il y a 2 solutions possibles pour réaliser cette transformation. Dans les 2 cas, la requête 4 se fera par l'intermédiaire d'un find. Même si avec la fonction aggregate est envisageable, il vaut mieux utiliser un find qui est plus rapide.

Solution 1 : utiliser des find et la fonction aggregate sur la collection machines.

Dans cette solution, la fonction aggregate est nécessaire pour construire les requêtes 1, 2 et 3.

Solution 2 : utiliser des find, la fonction aggregate et la fonction mapReduce en créant 2 collections supplémentaires, ce qui porte à 3 le nombre total de collection.

Dans cette solution, je crée à partir de la collection **machines** la collection **machines_logiciels**. Cette collection va me permettre de construire avec un simple find la requête 2 et avec une fonction aggregate la requête 3. A partir de la collection machines_logiciels je crée la collection **logiciels_nb**. Cette collection va me permettre de construire avec un simple find la requête 1.

La création de ces 2 collections supplémentaires s'effectue lors de l'exécution du script PHP d'importation des données.

Première étape, création de la collection machines_logiciels. Ci dessous, la fonction mapReduce que j'applique sur la collection machines.

```
var mapfunction = function() {
    var key=this.ordinateur.nom;
    var reseau=this.ordinateur.reseau;
    var date_inventaire=this.date_inventaire;
    var tab_soft=[];
    this.logiciels.forEach(function(e,i,a) {
        if(e.type==0) {
            var soft={};
            soft._id=e._id;
            soft.nom=e.nom;
            soft.date_installation=e.date_installation;
            soft.editeur=e.editeur;
            soft.version=e.version;
            tab_soft.push(soft);
        }
    });
    var nb_logiciels=tab_soft.length;
    emit(key,{reseau:reseau,date_inventaire:date_inventaire,nb_logiciels:nb_logiciels,logiciels:tab_soft});
};
var reducefunction = function(key,valeur) { return valeur; };
db.machines.mapReduce(mapfunction,reducefunction,{out:{replace:"machines_logiciels"}})
```

Dans la collection machines, pour chaque document, je récupère le nom de l'ordinateur qui sera la clé, le nom du réseau et la date d'inventaire. Je parcours le tableau contenant la liste de logiciels en ne conservant que les logiciels dont le type est égale à 0 (logiciel classique), j'exclus tout ce qui est patch de sécurité Microsoft (type=1). Enfin je crée une nouvelle valeur nb_logiciels qui m'indique le nombre de logiciels installés sur l'ordinateur. Je ne fais aucun traitement dans la fonction reduce.

Au final dans ma collection machines_logiciels, le canvas d'un document sera le suivant.

```

{
  _id:"nom de la machine",
  "value" : {
    "reseau":"nom du réseau",
    "date_inventaire":"date de l'inventaire",
    "nb_logiciels": nombre de logiciels installés sur ma machine,
    "logiciels": [
      {
        "_id":identifiant_logiciel,
        "nom":"nom_logiciel",
        "date_installation":"date_installation_logiciel",
        "editeur":"nom_editeur"
        "version":"version_logiciel"
      },
      ...
    ]
  }
}

```

Deuxième étape, création de la collection logiciels_nb. Ci-dessous, la fonction mapReduce que j'applique sur la collection machines_logiciels.

```

var mapfunction = function() {
  var reseau=this.value.reseau;
  this.value.logiciels.forEach(function(e,i,a) {
    var key={id:e._id,reseau:reseau};
    var valeurs={nom:e.nom,version:e.version,nb:1};
    emit(key,valeurs);
  });
};

var reducefunction= function(key, values) {
  var result={nb:0}
  values.forEach(function(e,i,a) {
    result.nom=e.nom;
    result.version=e.version;
    result.nb += e.nb;
  });
  return result;
};

db.machines_logiciels.mapReduce(mapfunction,reducefunction,{out:{replace:"logiciels_nb"}})

```

Dans la collection machines_logiciels, pour chaque document je parcours le tableau contenant les logiciels, puis j'émet, pour chaque logiciel, une paire clé valeur de la forme `{{id:identifiant,reseau:reseau}:{nom:logiciel,version:version,nb:1}}`. A la fin du traitement de la fonction map, les paires clés valeur seront de la forme `{{id:identifiant,reseau:reseau}:[{nom:logiciel,version:version,nb:1},{nom:logiciel,version:version,nb:1},...]}`

Ensuite la fonction reduce va parcourir la valeur de chaque élément généré par la fonction map `{{id:identifiant,reseau:reseau}:[{nom:logiciel,version:version,nb:1},{nom:logiciel,version:version,nb:1},...]}` en stockant dans une variable le nom, la version du logiciel, et en additionnant à chaque fois la valeur nb.

Au final dans ma collection logiciels_nb, le canvas d'un document sera le suivant.

```
{
  "_id":{
    "id":identifiant du logiciel
    "reseau":nom du réseau
  },
  "value": {
    "nb": nombre d'installation du logiciel
    "nom": nomdu logiciel
    "version": version du logiciel
  }
}
```

Voici l'adaptation des requêtes SQL en NoSQL pour les solutions 1 et 2.

Requête 1

```
select l.id as logiciel_id , v.id as version_id , l.nom as logiciel_nom, v.nom as
logiciel_version, COUNT(ml.id_logiciels) as nb_logiciels
from logiciels l
left join machines_logiciels ml on ml.id_logiciels=l.id
left join versions v on v.id=ml.versions_id
left join machines m on m.id=ml.id_machines
where l.etat=1 and l.type_id=0 and m.reseaux_id=0
group by l.id,v.id,l.nom,v.nom
order by logiciel_nom
```

Solution 1 :

```
db.machines.aggregate(
  {$match:{'ordinateur.reseau':'intra'}},
  {$project: {'logiciels':1}},
  {$unwind: '$logiciels'},
  {$group: {_id: {nom: "$logiciels.nom", version: "$logiciels.version", type: "$logiciels.type"}, nb: {$sum:1}}},
  {$match: {'_id.type':0}},
  {$sort: {'_id.nom':1}}
)
```

Solution 2 :

```
db.logiciels_nb.find({'_id.reseau':'intra'},{'_id':0}).sort({'value.nom':1})
```


Requête 2

```
select m.id as machine_id, m.nom as machine_nom, m.date_inventaire as date_inventaire,
COUNT(ml.id_logiciels) as nb_logiciels
from Machines m
left join Machines_Logiciels ml on ml.id_machines=m.id
left join Logiciels l on l.id=ml.id_logiciels
where m.reseaux_id=0 and l.type_id=0
group by m.id,m.nom,m.date_inventaire
```

Solution 1

```
db.machines.aggregate(
  {$match:{'ordinateur.reseau':'intra'}},
  {$project:{'date_inventaire':1,'ordinateur.nom':1,'logiciels._id':1,'logiciels.type':1}},
  {$unwind:'$logiciels'},
  {$match:{'logiciels.type':0}},
  {$group:{_id:{ordinateur:'$ordinateur.nom',inventaire:'$date_inventaire'},nb_logiciels:{$sum:1}}},
  {$sort:{'_id.ordinateur':1}}
)
```

Solution 2

```
db.machines_logiciels.find({"value.reseau":"intra"},"value.date_inventaire":1,"value.nb_logiciels":1})
```

Requête 3

```
select l.nom as logiciel_nom,ml.date_installation as date_installation,v.nom as
logiciel_version
from Logiciels l
left join Machines_Logiciels ml on ml.id_logiciels=l.id
left join Machines m on m.id=ml.id_machines
left join versions v on v.id=ml.versions_id
where m.id=330 and l.type_id=0
order by logiciel_nom
```

Solution 1

```
db.machines.aggregate(
  {$match:{'ordinateur.reseau':'intra','ordinateur.nom':'MVL-AUCLAIR'}},
  {$project:{'_id':0,'logiciels.nom':1,'logiciels.version':1,'logiciels.date_installation':1,'logiciels.type':1}},
  {$unwind:'$logiciels'},
  {$match:{'logiciels.type':0}},
  {$project:{'logiciels.nom':1,'logiciels.version':1,'logiciels.date_installation':1}},
  {$sort:{'logiciels.nom':1}}
)
```

Solution 2

```
db.machines_logiciels.aggregate({$match:{_id:"MVL-AUCLAIR"}},{$project:{'value.logiciels.nom':1,'value.logiciels.version':1,'value.logiciels.date_installation':1}},{$unwind:'$value.logiciels'},{$sort:{'value.logiciels.nom':1}})
```

Requête 4

```
select m.nom,ml.date_installation from Machines m
left join Machines_Logiciels ml on ml.id_machines=m.id
where ml.id_logiciels=264 and ml.versions_id=78 and m.reseaux_id=0
order by m.nom
```

Solution 1 et Solution 2 :

```
db.machines.find(
  {'ordinateur.reseau':'intra',"logiciels.nom" : "Minimal SYStem 1.0.11", "logiciels.version" : "1.0.11"},
  {
    '_id':0,
    logiciels:{$elemMatch: {'nom':"Minimal SYStem 1.0.11",'version':"1.0.11"}},
    'ordinateur.nom':1,
    'logiciels.date_installation':1
  }
).sort({'ordinateur.nom':1})
```

logiciels:{\$elemMatch: {'nom':"Minimal SYStem 1.0.11",'version':"1.0.11"}} permet de limiter l'affichage de la date d'installation uniquement pour le logiciel souhaité. Si cette condition n'est pas spécifiée dans la partie projection, la requete retournera la date d'installation de chaque logiciel.

Attention avec l'opérateur \$elemMatch, si plusieurs entrées dans le tableau correspondent aux critères de \$elemMatch, seul le premier est retourné.

Les tests de performance

Bien que ma base de données ne soit pas très volumineuse, j'ai réalisé quelques tests de performance sur 5 opérations

Importation des données

Ce test mesure le temps d'exécution du script PHP qui permet d'importer les données dans la base.

Recherche d'un logiciel

L'interface de mon application utilise le plugin JavaScript JQuery datable pour générer les divers tableaux. Dans ce plugin, lorsque l'utilisateur saisit quelque chose dans la barre de recherche, une requête est déclenchée à chaque caractère saisie. Le plugin génère une requête SQL utilisant un **like '%terme%'** pour obtenir les résultats. Ce test mesure le temps total pour faire une recherche.

Modification de l'affichage

L'utilisateur peut sélectionner le nombre d'éléments qu'il souhaite afficher par page. Par défaut il y a 25 éléments qui sont affichés. Ce test mesure le temps pour passer d'un affichage de 25 éléments à la totalité des éléments.

Tri sur le nom du logiciel

Ce test mesure le temps pour trier la totalité de la liste des logiciels.

Navigation dans les pages

Ce test mesure le temps pour passer d'une page à l'autre. Chaque page étant constituée de 50 éléments.

Résultats

Voici les résultats des tests effectués

	SQL	MongoDB solution 1	MongoDB solution 2
Importation des données	2 min 30 s	6,15 s	10,20 s
Recherche d'un logiciel	2,60 s	3 s	1,70 s
Modification de l'affichage	120 ms	405 ms	15 ms
Tri sur le nom du logiciel	140 ms	405 ms	15 ms
Navigation dans les pages	180 ms	405 ms	10 ms

Pour l'importation des données, MongoDB devance largement le SQL, ceci est dû au fait qu'en NoSQL il n'y a pas de schéma relationnel. En MongoDB, le script lie les fichiers xml, le transforme en format JSON et insert les documents dans la base. En SQL, le script lit les fichiers xml et retravaille les données pour qu'elles correspondent au schéma relationnel. La solution 1 est la plus rapide puisqu'elle remplit uniquement la collection machines tandis que la solution 2, en plus de remplir la collection machines, crée les collections machines_logiciels et logiciels_nb.

Pour la recherche d'un logiciel, les temps sont assez proche, la solution 1 est la plus lente, ceci est dû à l'enchaînement des fonctions dans le pipeline de la fonction aggregate, la solution 2 utilise juste une fonction find, elle est très rapide. En SQL, c'est entre les solutions 1 et 2.

Pour les 3 autres tests, la solution 2 est très rapide et devance assez largement la solution 1 et le SQL. La fonction aggregate de MongoDB est donc à éviter. Le fait de construire plusieurs collections et d'essayer d'utiliser au maximum des fonctions find fait gagner en performance mais prendra plus de place au niveau du stockage.

Conclusion

Intégrer MongoDB dans mon architecture de départ n'est pas très compliqué grâce au pilote PHP fourni avec MongoDB. La transformation de la base SQL en NoSQL n'est pas forcément facile, il faut complètement sortir de la logique relationnelle et s'adapter à l'environnement NoSQL.

Bien qu'une caractéristique du NoSQL soit de ne pas avoir de schéma de données prédéfini, pour pouvoir réaliser des traitements sur une collection, il faut avoir un minimum des données structurées sinon cela devient vite ingérable. Dans mon cas tous les documents ont tous la même structure.

Dans l'application que j'ai transformée il y a essentiellement des interrogations de la base, le NoSQL semble plus performant que le SQL dans ce cas-là, reste à voir son comportement dans une application avec beaucoup de transactions.

Finalement, ce projet m'a permis de découvrir les bases de données NoSQL que je ne connaissais pas du tout. Ce type de base de données pourra me servir en vue de développer une application d'analyse de logs serveur.